LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK (OOP)

SISTEM PENGELOLAAN DATA POS, GUDANG, PELANGGAN RITEL



Disusun Oleh (Kelompok 2):

Muhamad Wildan Muzaqi	**225443014**
Josua Pardamean Simangunsong	**225443011**
Muhammad Dhafin Rizqullah	**225443015**
Dhia Athalla	**225443006**
Alexander Dwiky Paskahary Putra	**225443003**

Dosen Pengampu: Mohammad Harry Khomas Saputra, S.T., M.TI

2 AEC-1

TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI

POLITEKNIK MANUFAKTUR BANDUNG

TAHUN AJARAN 2026/2027

1.1 Cerita Kebutuhan (Spesifikasi Fungsional)

1.1.1 Latar Belakang

Ritel membutuhkan sistem terintegrasi untuk mengelola transaksi kasir, produk, persediaan gudang, pelanggan, karyawan, pembayaran, dan laporan penjualan. Pencatatan manual berisiko menimbulkan selisih stok, kesalahan perhitungan, duplikasi data, serta kesulitan melacak riwayat transaksi. Proyek pada repositori pbo_2 mengarah pada aplikasi POS berbasis web dengan arsitektur client–server dan prinsip Object-Oriented Programming (OOP).

1.1.2 Aktor Sistem

Aktor utama adalah Manager dan Kasir. Manager mengelola master produk, stok gudang, data pelanggan, akun/role, dan laporan. Kasir berfokus pada pencarian atau pemindaian barcode, pengelolaan keranjang, pembayaran, dan pencetakan struk. Pelanggan menjadi entitas data yang dapat terhubung dengan program loyalitas/member.

1.1.3 Kebutuhan Fungsional

Sistem perlu: (1) autentikasi dan pembatasan akses berdasarkan role; (2) mengelola data produk dan harga; (3) mencatat stok masuk/keluar serta audit pergerakan stok; (4) memberi peringatan stok minimum; (5) mencari produk berdasarkan nama/kode/barcode; (6) membangun keranjang transaksi; (7) menghitung subtotal, diskon, dan PPN 11% sesuai aturan proyek; (8) mendukung pembayaran tunai, QRIS, dan debit; (9) menyimpan transaksi dan detailnya; (10) menjaga snapshot harga pada saat transaksi; (11) mengelola pelanggan/member dan diskon loyalitas; (12) menghasilkan laporan penjualan/omzet; dan (13) mencetak struk yang rapi.

1.1.4 Kebutuhan Nonfungsional

Aplikasi sebaiknya mudah digunakan, memisahkan antarmuka dari aturan bisnis, menjaga konsistensi transaksi, memiliki konfigurasi yang dapat dipindahkan antarl ingkungan, menyediakan pengujian otomatis, dan tidak menyimpan rahasia seperti password database di repositori. Komunikasi frontend–backend menggunakan REST API berbasis JSON.

1.2 Dari Kebutuhan ke Desain: Menemukan Class

1.2.1 Identifikasi Entitas

Entitas domain yang relevan meliputi Product/Item, InventoryMovement, Cart, CartItem, Sale/Transaction, SaleLine, Payment, Customer/Member, Employee, Manager, Cashier, User/Role, serta Report. Nama class final harus mengikuti implementasi aktual di folder serverSide dan client/src.

1.2.2 Relasi Class

Product menjadi data barang yang dijual dan disimpan. InventoryMovement mencatat perubahan stok. Cart menampung CartItem sebelum transaksi diselesaikan. Sale memiliki satu atau lebih SaleLine dan satu atau lebih informasi pembayaran sesuai aturan yang diterapkan. Customer dapat dikaitkan dengan transaksi atau program loyalitas. Manager dan Cashier merupakan jenis Employee/User dengan hak akses berbeda.

1.2.3 Pemetaan dari Contoh Perpustakaan ke Ritel

Contoh buku perpustakaan pada modul pembelajaran tidak diterapkan secara literal. Konsep abstract class Item dipetakan ke abstraksi produk/barang; subclass Buku, Majalah, dan DVD diganti dengan kategori/jenis produk ritel bila memang membutuhkan perilaku berbeda. Anggota dipetakan menjadi Customer/Member, sedangkan Peminjaman dipetakan secara konseptual ke Sale/Transaction. Interface pencarian diterapkan sebagai kontrak pencarian produk/pelanggan, bukan pencarian buku.

1.2.4 Prinsip Desain

Pisahkan domain/model, service/aturan bisnis, repository/akses data, DTO/kontrak API, routes/controller, dan tampilan. Dengan pemisahan ini, perubahan tampilan tidak perlu mengubah perhitungan harga atau aturan stok.

1.3 Persiapan Alat

1.3.1 Perangkat Lunak

Berdasarkan README repositori, jalur Docker menggunakan Docker Desktop dan Docker Compose. Jalur lokal memerlukan JDK 21 atau lebih baru serta PHP 8 atau lebih baru. Backend menggunakan Kotlin dan Ktor, frontend menggunakan PHP 8.2 dengan pola MVC, dan basis data utama menggunakan PostgreSQL 17. Backend menyediakan fallback H2 in-memory untuk pengembangan lokal jika konfigurasi database produksi tidak tersedia.

1.3.2 Perangkat Pendukung

Gunakan Git, browser modern, editor kode seperti IntelliJ IDEA/Android Studio atau VS Code, terminal, dan alat pengujian API seperti Postman/Postman CLI. Pastikan port 8000, 8080, dan 5432 tidak sedang dipakai aplikasi lain.

1.3.3 Persiapan Keamanan

Salin .env.example menjadi .env untuk konfigurasi lokal dan ganti kredensial contoh sebelum penggunaan bersama. Jangan commit file .env ke Git. Kredensial demo di README hanya untuk pengujian lokal dan tidak boleh digunakan sebagai akun produksi.

1.4 Langkah 1—Menyiapkan Proyek

1.4.1 Memperoleh Kode

Clone repositori https://github.com/aroodoleoon/pbo_2, lalu masuk ke direktori pbo_2. Periksa README.md, struktur client/, serverSide/, database/, docs/, compose.yaml, dan docker-compose.yml. Perlu diperhatikan bahwa README dan struktur yang tampil di halaman repositori merupakan sumber panduan; bila branch/commit berubah, cocokkan nama folder dengan versi yang sedang digunakan.

1.4.2 Memahami Struktur

client/ berisi frontend PHP, front controller/routing, model, controller, session, dan views. serverSide/ berisi backend Kotlin/Ktor, domain, service, repository, DTO, dan routes. database/ berisi migrasi skema dan data awal. docs/ menyimpan dokumentasi API, kontrak, panduan menjalankan, dan koleksi pengujian API.

1.4.3 Menetapkan Konfigurasi

Buat konfigurasi lokal dari .env.example, atur password yang aman, lalu periksa konfigurasi pada Compose dan backend. Jangan menaruh kredensial langsung di source code. Pastikan konfigurasi client menunjuk ke alamat backend yang benar.

1.5 Langkah 2—Abstract Class Item

1.5.1 Konsep

Abstract class digunakan untuk menyimpan atribut dan perilaku umum dari beberapa jenis objek. Pada sistem ritel, konsep ini dapat dimodelkan sebagai Product atau Item jika seluruh barang memiliki atribut umum seperti id, nama, kode/barcode, harga, dan status aktif. Class abstrak tidak dibuat menjadi objek langsung; objek konkret dibuat dari class turunan atau class produk konkret.

1.5.2 Contoh Konseptual Kotlin

Contoh berikut menjelaskan ide desain, bukan klaim bahwa nama class ini persis sudah ada di repositori:

```
abstract class Item(
    val id: String,
    val name: String,
    val barcode: String,
    val basePrice: Long
) {
    abstract fun categoryLabel(): String
    fun isValidPrice(): Boolean = basePrice >= 0
}
```

1.5.3 Penerapan

Gunakan abstraksi hanya jika ada perilaku bersama yang nyata. Jika produk cukup dibedakan oleh atribut category, pendekatan satu class Product dengan enum/kategori bisa lebih sederhana daripada membuat banyak subclass tanpa perilaku berbeda. Harga transaksi tetap perlu disalin sebagai snapshot pada saat checkout agar perubahan harga master tidak mengubah transaksi lama.

1.6 Langkah 3—Subclass Buku, Majalah, dan DVD

1.6.1 Adaptasi ke Domain Ritel

Pada contoh perpustakaan, Buku, Majalah, dan DVD adalah subclass Item. Dalam sistem ritel, subclass dapat berupa GroceryProduct, ElectronicProduct, atau ClothingProduct jika masing-masing memiliki aturan yang berbeda. Jangan membuat subclass hanya karena nama kategori berbeda; gunakan subclass saat perilaku atau validasi memang berbeda.

1.6.2 Polimorfisme

Polimorfisme memungkinkan kode memanggil operasi umum pada objek dari tipe dasar, sementara implementasi khusus ditentukan oleh jenis objek. Misalnya, produk tertentu dapat memiliki aturan garansi atau masa kedaluwarsa yang berbeda. Implementasi aktual perlu diselaraskan dengan class domain di serverSide/src/main/kotlin/com/pos/.

1.6.3 Validasi

Pastikan harga tidak negatif, kode/barcode tidak duplikat jika kebijakan mengharuskannya, dan stok tidak menjadi negatif. Validasi aturan bisnis harus berada di lapisan domain/service, bukan hanya mengandalkan validasi form pada frontend.

1.7 Langkah 4—Data Class Anggota dan Peminjaman

1.7.1 Customer/Member

Padanan Anggota adalah Customer atau Member. Data yang umum diperlukan antara lain id pelanggan, nama, nomor kontak, tanggal bergabung, status keanggotaan, dan poin/tingkat loyalitas bila digunakan. Hindari menyimpan data pribadi yang tidak dibutuhkan.

1.7.2 Sale/Transaction

Padanan konseptual Peminjaman adalah Sale/Transaction. Transaksi mencatat nomor transaksi, waktu, kasir, pelanggan opsional, daftar item, subtotal, diskon, pajak, total akhir, status, dan informasi pembayaran. Detail transaksi menyimpan kuantitas dan harga satuan saat transaksi.

1.7.3 Konsistensi Data

Selesaikan transaksi secara atomik: pencatatan transaksi, detail, pembayaran, dan pengurangan stok harus berhasil sebagai satu kesatuan atau dibatalkan bila terjadi kegagalan. Aturan ini mencegah transaksi tercatat tetapi stok tidak berubah, atau stok berkurang tanpa transaksi.

1.8 Langkah 5—Interface Pencarian

1.8.1 Kontrak Pencarian

Interface mendefinisikan operasi yang harus disediakan class pelaksana. Pada sistem ritel, kontrak dapat berupa ProductSearch dengan operasi pencarian berdasarkan nama, SKU, atau barcode. Implementasi pencarian dapat menggunakan repository/database sehingga antarmuka tidak bergantung pada detail penyimpanan.

1.8.2 Contoh Konseptual

```
interface Searchable<T> {
  fun search(query: String): List<T>
}
```

Konsep ini hanya ilustrasi. Sesuaikan generic type, nama method, dan pola coroutine dengan gaya implementasi Ktor yang benar-benar dipakai.

1.8.3 Hasil Pencarian

Pencarian harus menangani input kosong, hasil tidak ditemukan, banyak hasil yang mirip, dan barcode yang cocok tepat. Untuk kasir, hasil barcode yang tepat sebaiknya dapat langsung ditambahkan ke keranjang dengan pemeriksaan stok.

1.9 Langkah 6—Custom Exception

1.9.1 Tujuan

Custom exception membuat kegagalan aturan bisnis lebih jelas daripada pesan error generik. Contoh: ProductNotFoundException, InsufficientStockException, InvalidPaymentException, DuplicateBarcodeException, atau UnauthorizedActionException.

1.9.2 Penanganan

Service melempar exception ketika aturan dilanggar; lapisan API menerjemahkannya menjadi status HTTP dan respons JSON yang konsisten. Contoh umum: data tidak ditemukan menjadi 404, input tidak valid menjadi 400/422, tidak terautentikasi menjadi 401, dan tidak berwenang menjadi 403. Jangan mengirim stack trace atau kredensial ke pengguna.

1.9.3 Batas Transaksi

Jika pembayaran gagal, transaksi database harus di-rollback agar stok dan catatan penjualan tidak berada dalam keadaan setengah jadi. Catat rincian teknis ke log server tanpa membocorkan informasi sensitif.

1.10 Langkah 7—Class Utama Perpustakaan

1.10.1 Padanan Pengelola Sistem

Class Perpustakaan pada contoh tutorial berperan sebagai koordinator operasi. Dalam sistem ritel, tanggung jawab serupa sebaiknya dipisahkan menjadi service seperti InventoryService, CheckoutService, CustomerService, PaymentService, dan ReportService, bukan ditumpuk dalam satu class besar.

1.10.2 Tanggung Jawab

InventoryService mengatur pergerakan stok dan batas minimum. CheckoutService memvalidasi keranjang, menghitung harga/diskon/pajak, dan membuat transaksi. PaymentService memvalidasi metode pembayaran. CustomerService mengelola pelanggan/member. ReportService mengagregasi data untuk laporan.

1.10.3 Repository

Repository mengisolasi akses database dari aturan bisnis. Service bergantung pada abstraksi repository sehingga pengujian dapat memakai data tiruan dan implementasi database dapat diganti dengan dampak lebih kecil.

1.11 Langkah 8—Antarmuka CLI (Main.kt)

1.11.1 Perbedaan dengan Proyek

Tutorial perpustakaan memakai CLI melalui Main.kt. Repositori pbo_2 yang dianalisis justru dirancang sebagai aplikasi web client–server: frontend PHP dan backend Kotlin/Ktor. Oleh karena itu, Main.kt berbasis CLI bukan antarmuka utama proyek ini. Titik masuk backend dijalankan melalui Gradle/Ktor, sementara antarmuka pengguna dibuka lewat browser.

1.11.2 Alur Antarmuka

Pengguna login, sistem memeriksa identitas dan role, lalu menampilkan modul sesuai hak akses. Kasir memilih/scan produk, mengatur jumlah, melihat total, memilih metode pembayaran, menyelesaikan transaksi, dan mencetak struk. Manager mengelola produk, stok, pelanggan, dan laporan.

1.11.3 Kontrak API

Frontend mengirim request HTTP dengan JSON ke endpoint backend. Backend memvalidasi request, menjalankan service/domain, menyimpan data, lalu mengembalikan response terstruktur. Detail endpoint harus mengikuti docs/API_DOCUMENTATION.md dan docs/API_CONTRACT.md di versi repositori yang digunakan.

1.12 Langkah 9—Menjalankan dan Menguji Manual

6.12.1 Menjalankan dengan Docker Compose

Dari direktori root proyek, jalankan:

docker compose up --build -d

docker compose ps

README menyebut tiga layanan utama: PostgreSQL, backend, dan client. Pastikan layanan berstatus berjalan/healthy sebelum membuka aplikasi.

1.12.2 Akses Aplikasi

Menurut README repositori, client tersedia di http://localhost:8000 dan API di http://localhost:8080/api/v1. Database PostgreSQL memakai port 5432. Jangan mengekspos database ke jaringan publik untuk penggunaan produksi tanpa konfigurasi keamanan yang memadai.

1.12.3 Menjalankan Lokal Tanpa Docker

Backend: buka terminal di root proyek, masuk ke serverSide, lalu jalankan .\gradlew.bat run pada Windows. Frontend: buka terminal kedua dan jalankan php -S 127.0.0.1:8000 -t client. Gunakan JDK 21+ dan PHP 8+ sesuai panduan README.

1.12.4 Skenario Uji Manual

Uji login Manager dan Kasir; pastikan role membatasi menu; cari produk melalui nama dan barcode; tambahkan produk ke keranjang; ubah kuantitas; periksa diskon dan PPN; coba stok tidak cukup; selesaikan pembayaran tunai/QRIS/debit; periksa stok setelah transaksi; cetak struk; cari pelanggan; dan periksa laporan penjualan. Catat hasil aktual sebagai Lulus/Gagal dan sertakan bukti layar.

1.13 Langkah 10—Unit Test

1.13.1 Backend Kotlin

README memberikan perintah dari folder serverSide: .\gradlew.bat test. Pengujian domain yang disebutkan mencakup enkapsulasi barang, pewarisan karyawan/manager/kasir, polimorfisme diskon member, serta snapshot harga transaksi. Jalankan tes pada commit yang sama dengan kode yang dilaporkan dan simpan output aktual.

1.13.2 Pengujian Modul Lain

README juga menyebut pengujian inventaris Python dengan python -m unittest discover tests pada direktori modul inventaris, serta pengujian REST API menggunakan Postman CLI. Namun lokasi modul yang tertulis pada README perlu dicocokkan dengan struktur folder yang benar-benar ada di branch terbaru sebelum perintah dijalankan.

1.13.3 Kasus Uji Penting

Tambahkan tes untuk stok cukup/tidak cukup, barcode tidak ditemukan/duplikat, kuantitas nol atau negatif, pembayaran kurang, diskon member, pembulatan pajak, transaksi dibatalkan, rollback ketika pembayaran gagal, dan perubahan harga setelah transaksi. Jangan menuliskan bahwa seluruh tes lulus sebelum benar-benar dijalankan.

1.14 Langkah 11—Docker (Bonus)

1.14.1 Tujuan

Docker mengemas aplikasi dan dependensinya dalam kontainer sehingga lingkungan pengembangan lebih konsisten. Compose mengatur layanan database, backend, dan frontend beserta jaringan dan konfigurasi yang diperlukan.

1.14.2 Perintah Dasar

Jalankan docker compose up --build -d untuk membangun dan menjalankan layanan, docker compose ps untuk memeriksa status, docker compose logs untuk melihat log, dan docker compose down untuk menghentikan layanan. Gunakan nama file Compose yang memang menjadi konfigurasi resmi di root.

1.14.3 Catatan Keamanan

Gunakan secret atau environment variable untuk kredensial, ganti password contoh, jangan memasukkan .env ke Git, dan jangan menganggap port database harus terbuka ke jaringan umum. Untuk produksi, tambahkan backup, health check, pembatasan akses, dan strategi migrasi database.

1.15 Langkah 12—Publikasi ke GitHub

1.15.1 Alur Git

Gunakan branch untuk pekerjaan fitur, commit dengan pesan yang jelas, lakukan pull/rebase sesuai kesepakatan tim, lalu buka pull request untuk review sebelum merge. Setiap perubahan skema database harus disertai migration dan dokumentasi.

1.15.2 Berkas yang Dipublikasikan

Sertakan source code, README, dokumentasi API, migration, seed data non-sensitif, contoh konfigurasi .env.example, dan petunjuk menjalankan serta menguji. Jangan mengunggah .env, token, password produksi, dump database berisi data pribadi, atau file build yang tidak dibutuhkan.

1.15.3 Repositori Proyek

Repositori yang dianalisis adalah https://github.com/aroodoleoon/pbo_2. README mencantumkan pembagian modul tim: Alex (inventaris dan audit stok), Dapin (transaksi POS dan snapshot keranjang), Dhia (karyawan, autentikasi, loyalitas), dan Josua (pembayaran, database, laporan). Konfirmasikan kembali pembagian ini dengan anggota tim sebelum dijadikan daftar penulis final.

1.16 Ringkasan Pilar OOP pada Sistem Pengelolaan Data POS, Gudang, Pelanggan di Ritel

Enkapsulasi

Data penting seperti harga, stok, status transaksi, dan hak akses tidak boleh diubah sembarangan. Perubahan dilakukan melalui method/service yang memvalidasi aturan bisnis.

Abstraksi

Class dan interface menampilkan operasi yang dibutuhkan, sementara detail database, perhitungan, dan komunikasi jaringan disembunyikan di balik service/repository.

Pewarisan

Perilaku umum dapat ditempatkan pada class dasar, misalnya Employee, lalu diturunkan menjadi Manager dan Cashier jika struktur aktual memang memerlukan pewarisan.

Polimorfisme

Operasi yang sama dapat memiliki perilaku berbeda, contohnya perhitungan diskon untuk level member yang berbeda atau strategi pembayaran yang berbeda. Pilih desain yang tetap mudah diuji dan tidak membuat hierarki berlebihan.

Komposisi

Sale terdiri dari beberapa SaleLine; Cart terdiri dari CartItem; layanan checkout menggunakan payment dan inventory service. Komposisi sering lebih sesuai daripada pewarisan untuk hubungan 'memiliki'.

Pemisahan Tanggung Jawab

MVC di frontend dan pemisahan domain/service/repository/routes di backend membantu menjaga kode tetap modular, dapat diuji, dan mudah dikembangkan.

1.17 Latihan Pengembangan

Latihan 1—Produk

Tambahkan pencarian produk berdasarkan barcode dan nama, termasuk kondisi tidak ditemukan. Sertakan unit test untuk input kosong dan barcode duplikat.

Latihan 2—Stok

Buat notifikasi stok minimum dan histori barang masuk/keluar. Pastikan perubahan stok selalu mencatat alasan, waktu, dan pengguna yang melakukan perubahan.

Latihan 3—Checkout

Uji perhitungan subtotal, diskon bertingkat, PPN 11%, dan pembulatan. Bandingkan hasil UI dengan hasil service pada kasus batas.

Latihan 4—Pelanggan

Tambahkan riwayat transaksi pelanggan dan aturan poin loyalitas. Tentukan aturan poin untuk transaksi yang dibatalkan atau direfund.

Latihan 5—Pembayaran

Rancang interface strategi pembayaran untuk Cash, QRIS, dan Debit. Validasi jumlah yang dibayar dan pastikan transaksi tidak selesai sebelum pembayaran terkonfirmasi.

Latihan 6—Keandalan

Buat simulasi kegagalan ketika penyimpanan pembayaran dilakukan. Pastikan transaksi dan perubahan stok di-rollback.

Latihan 7—Dokumentasi

Perbarui diagram class, ERD, API contract, dan panduan instalasi ketika struktur atau endpoint berubah.

1.18 Masalah yang Sering Muncul dan Solusinya

Docker tidak berjalan

Pastikan Docker Desktop aktif, virtualisasi tersedia, dan perintah dijalankan dari direktori yang berisi file Compose. Periksa docker compose ps serta docker compose logs.

Port sudah digunakan

Cari proses yang memakai port 8000, 8080, atau 5432, lalu hentikan proses tersebut atau ubah port mapping dan konfigurasi client.

Backend gagal start

Periksa JDK dengan java -version, wrapper Gradle, konfigurasi environment, log aplikasi, dan koneksi database. Pastikan nama folder backend sesuai dengan versi repo.

Frontend tidak terhubung ke API

Pastikan backend aktif, URL API sesuai, dan request/response mengikuti kontrak. Periksa console browser, log PHP, status HTTP, dan konfigurasi CORS jika lintas origin.

Database gagal konek

Verifikasi service PostgreSQL, kredensial, nama database, port, migration, dan environment. Untuk mode lokal, periksa apakah fallback H2 memang aktif pada konfigurasi tersebut.

Stok menjadi negatif

Validasi stok pada service dan lakukan update stok dalam transaksi database. Hindari hanya memvalidasi stok pada tampilan karena permintaan paralel dapat menyebabkan race condition.

Total transaksi berbeda

Gunakan satu sumber aturan perhitungan pada backend, tetapkan urutan diskon/pajak, aturan pembulatan, dan snapshot harga. Jangan mempercayai total yang hanya dikirim oleh browser.

Tes gagal

Jalankan satu test yang gagal, baca stack trace, periksa fixture/seed, zona waktu, dan ketergantungan antar-test. Jangan mengubah assertion hanya agar tes tampak lulus.

Rahasia masuk Git

Cabut/rotasi kredensial yang terlanjur terunggah, hapus dari riwayat bila perlu, dan pindahkan konfigurasi ke environment variable. Menghapus file pada commit terbaru saja tidak membuat secret lama aman.

Dokumentasi berbeda dengan kode

Periksa commit/branch aktif, perbarui README dan API contract, dan catat hasil pengujian berdasarkan versi yang benar. Nama folder dan command dari dokumentasi lama bisa berubah.

Referensi

1. Repositori proyek pbo_2, README dan struktur direktori, https://github.com/aroodoleoon/pbo_2 (diakses 15 Oktober 2026).

2. Analisis dan Pembagian Tim Proyek POS OOP, file PDF yang tercantum pada root repositori pbo_2.

3. Dokumentasi internal repositori: docs/API_DOCUMENTATION.md, docs/API_CONTRACT.md, docs/HOW_TO_RUN.md, serta file konfigurasi Compose (sesuaikan dengan branch/commit yang digunakan).